



BOOK THREE

Frontend Complete Guide

The React and Vite browser application, explained file by file

All paths are relative to `frontend/`.

One rule to keep in mind throughout: the frontend is a *convenience layer*. Hiding a button, redirecting a route, or validating a form makes the app pleasant to use — none of it is security. Every rule is enforced again by the backend. See *Backend Guide* §5.3.

CONTENTS



1	Vite + React startup	1	7	Pages vs components	14
	What Vite does	1	8	Shared components	15
	src/main.jsx — the entry point	1	9	Student flows	16
	src/App.jsx	2	10	Provider flows	17
2	React concepts, with RuWork examples	3	11	Admin workspace	18
	Components	3	12	Messaging UI	19
	JSX	3	13	Notifications UI	20
	Props	3	14	Styling with Tailwind	21
	State — useState	3		The design tokens	21
	Controlled inputs	4		Responsive prefixes — mobile-first	21
	Side effects — useEffect	4	15	Accessibility	23
	Memoisation — useMemo and useCallback	5		Labels	23
	Context	5		aria-invalid and aria-describedby	23
	Conditional rendering	6		aria-live	23
	Event handlers	6		Focus management	23
3	Routing	7		Skip link	24
	Route groups in App.jsx	7		Keyboard and semantics	24
	:id route parameters	7	16	Lazy loading and performance	25
	ProtectedRoute	8		The problem	25
4	Authentication on the frontend	9		The fix	25
	Storage — utils/authStorage.js	9		What stays eager, and why	25
	Reading the token — utils/token.js	9		The measured result	26
	AuthContext	10	17	Frontend testing	27
5	Axios and the shared API client	11		The tools	27
	Request interceptor — attaching the token	11		The guiding principle	27
	Response interceptor — handling revoked sessions	12		Shared setup	27
	Safe error messages — utils/apiError.js	12		Mocking services	28
6	The service layer	13		A representative test	28
				The suites	29
				Current totals	29
			18	File-by-file reference	30

Entry, config, routing	30
Context, hooks, utils	30
Services	31

Layout and shared	31
components	
Pages	32
Frontend common questions	33

Vite + React startup

What Vite does

Vite is two tools in one:

- **In development** (`npm run dev`) it serves your source files almost unchanged, using the browser's native ES modules. Saving a file updates the page in milliseconds.
- **In production** (`npm run build`) it bundles, minifies, and splits everything into optimised files in `dist/`.

`vite.config.js` configures three things:

```
export default defineConfig({  
  plugins: [react(), tailwindcss()],  
  server: { proxy: { "/api": { target: "http://localhost:5000", changeOrigin: true } } },  
  test: { environment: "jsdom", setupFiles: "./src/test/setup.js", css: true }  
});
```

JS

The dev proxy is worth understanding. In development the browser page is served from `localhost:5173` while the API runs on `localhost:5000` — two different origins, which normally triggers CORS. The proxy makes the frontend call its *own* origin at `/api/...`, and Vite forwards it to port 5000 behind the scenes. The browser only ever sees one origin, so CORS never comes up during development. In production `VITE_API_BASE_URL` is set to the real API URL and the backend's CORS allowlist takes over.

`src/main.jsx` — the entry point

```
createRoot(document.getElementById("root")).render(  
  <StrictMode>  
    <BrowserRouter>  
      <ToastProvider>  
        <AuthProvider>  
          <App />  
        </AuthProvider>  
      </ToastProvider>  
    </BrowserRouter>  
  </StrictMode>  
);
```

JSX

Read it outside-in — each wrapper provides something to everything inside it:

WRAPPER	PROVIDES
StrictMode	Development-only extra checks that surface unsafe patterns
BrowserRouter	URL awareness — without it <code>useNavigate</code> and <code><Link></code> cannot work
ToastProvider	The <code>showToast()</code> function to every component
AuthProvider	Login state to every component
App	The route map

`AuthProvider` is *inside* `ToastProvider` so authentication code can raise toasts.

src/App.jsx

`App` renders the route table and nothing else. Its structure:

```

<ScrollToTop />
<a href="#main-content" className="sr-only focus:not-sr-only ...">Skip to main content</a>
<div id="main-content" tabIndex={-1}>
  <Suspense fallback={<Spinner label="Loading RuWork..." />}>
    <Routes>...</Routes>
  </Suspense>
</div>

```

`ScrollToTop` resets scroll position on navigation (otherwise clicking a job from halfway down a list opens the details page already scrolled). The skip link and `Suspense` are covered in [§15](#) and [§16](#).

React concepts, with RuWork examples

Components

A component is a function that returns JSX. It must start with a capital letter — that is how JSX distinguishes `<Button />` (your component) from `<button />` (the HTML element).

```
// components/common/Spinner.jsx
export default function Spinner({ label = "Loading RuWork" }) {
  return (
    <div className="grid min-h-[45vh] place-items-center px-5" role="status">
      ...<p className="mt-4 text-sm font-semibold text-ink-600">{label}</p>
    </div>
  );
}
```

JSX

JSX

JSX looks like HTML but is JavaScript. Differences that trip people up:

- `class` → `className`
- `for` → `htmlFor`
- `{}` embeds a JavaScript expression: `{label}`, `{count > 0 ? "..." : null}`
- Attributes are camelCase: `aria-invalid` stays hyphenated (ARIA is the exception), but `onClick`, `tabIndex`, `maxLength` are camelCase.

Props

Props are the inputs a parent passes down. They are **read-only** — a child never modifies its props.

```
<Spinner label="Loading Admin dashboard..." />
```

JSX

`{ label = "Loading RuWork" }` in the signature is destructuring with a default value.

State — `useState`

State is data that changes over time and triggers a re-render when it does.

```
// pages/auth/LoginPage.jsx
const [form, setForm] = useState({ email: "", password: "" });
const [errors, setErrors] = useState({});
const [isSubmitting, setIsSubmitting] = useState(false);
```

JSX

`useState` returns a pair: the current value and a setter. Calling the setter re-renders the component.

Never mutate state directly. `form.email = "x"` does nothing visible because React compares by reference and sees the same object. Always create a new one:

```
setForm((current) => ({ ...current, email: event.target.value }));
```

JS

The `...current` spread copies the existing fields and the new key overrides one.

Passing a *function* to the setter (`setForm(current => ...)`) is the safe pattern when the new value depends on the old one.

Controlled inputs

RuWork uses **controlled** inputs: React state is the single source of truth, and the input reflects it.

```
<FormField
  id="student-email"
  value={form.email}
  onChange={(event) => setForm((current) => ({ ...current, email: event.target.value })))}
/>
```

JSX

The flow is: user types → `onChange` fires → state updates → re-render → input shows the new value. It feels circular but it means the state and the screen can never disagree.

Side effects — `useEffect`

`useEffect` runs code *after* rendering — fetching data, subscribing to events. It is how RuWork loads server data.

```
// pages/admin/AdminAuditTrailPage.jsx
useEffect(() => {
  let active = true;
  adminService.getAudits({ ...filters, page, limit: 20 })
    .then((data) => active && setState({ status: "success", audits: data.audits, ... }))
    .catch((error) => active && setState({ status: "error", ... }));
  return () => { active = false; };
}, [filters, page, retry]);
```

JSX

Three parts:

1. **The effect body** — what to do.
2. **The cleanup function** (`return () => ...`) — runs before the next effect and when the component unmounts.
3. **The dependency array** (`[filters, page, retry]`) — re-run only when one of these changes. An empty `[]` means “once on mount”; omitting it entirely means “after every render”, which is almost always a bug (an infinite fetch loop).

The active flag is important. If the user navigates away while the request is in flight, the component unmounts but the promise still resolves. Calling `setState` on an unmounted component is a memory leak and a React warning. The cleanup sets `active = false`, so the late response is ignored. This pattern appears in nearly every RuWork page — recognise it.

Memoisation — `useMemo` and `useCallback`

```
// context/AuthContext.jsx
const value = useMemo(() => ({
  isAuthenticated: Boolean(auth?.token), token: auth?.token || null,
  user: auth?.user || null, login, logout, replaceToken
}), [auth, isRestoring, login, logout, replaceToken]);
```

J S X

Without `useMemo`, a new object would be created on every render, and every component consuming the context would re-render even when nothing meaningfully changed. `useCallback` does the same for functions.

Do not sprinkle these everywhere. They are worth it for context values (which fan out widely) and for functions passed into dependency arrays.

Context

Context passes data down without threading props through every intermediate component. Without it, `AppHeader` would need `auth` passed from `App` through several layers (“prop drilling”).

RuWork has two: `AuthContext` and `ToastContext`. Each is split across two files:

- `context/authContextValue.js` — `createContext(null)`, the context object itself
- `context/AuthContext.jsx` — `AuthProvider`, the component holding the state
- `hooks/useAuth.js` — the consumer hook


```
export default function useAuth() {
  const value = useContext(AuthContext);
  if (!value) throw new Error("useAuth must be used inside AuthProvider");
  return value;
}
```

JS

Why a custom hook rather than calling `useContext` directly? The guard turns “why is `auth` undefined?” into an explicit error naming the cause. The two-file split exists because Vite’s Fast Refresh works best when a file exports only components.

Conditional rendering

```
{state.status === "loading" ? <Spinner label="Loading Jobs..." /> : null}
{state.error ? <Alert>{state.error}</Alert> : null}
{state.status === "success" && !state.jobs.length ? <EmptyState /> : null}
```

JSX

`&&` returns the right side only when the left is truthy. `?` : chooses between two.

⚠ Watch out for 0. `{count && <Badge/>}` renders a literal 0 when count is 0, because 0 is falsy but still renderable. RuWork’s `UnreadBadge` avoids this with an explicit comparison: `count > 0 ? <span...> : null`.

Event handlers

```
async function handleSubmit(event) {
  event.preventDefault(); // stop the browser's full-page form submit
  if (!validate()) return;
  setIsSubmitting(true);
  try { ... } finally { setIsSubmitting(false); }
}
```

JSX

`event.preventDefault()` is essential — without it the browser reloads the page and React state is lost. The `finally` guarantees the button is re-enabled even if the request fails.

Routing

React Router keeps the URL and the visible component in sync **without a page reload**.

Route groups in App.jsx

```
JSX
<Routes>
  { /* PUBLIC */ }
  <Route path="/" element={<LoginPage />} />
  <Route path="/login/student" element={<LoginPage role="student" />} />
  <Route path="/jobs" element={<FindJobsPage />} />
  <Route path="/jobs/:id" element={<JobDetailsPage />} />
  <Route path="/reset-password" element={<ResetPasswordPage />} />

  { /* ANY SIGNED-IN ROLE */ }
  <Route element={<ProtectedRoute allowedRoles={ANY_SIGNED_IN_ROLE} />}>
    <Route path="/account/password" element={<ChangePasswordPage />} />
  </Route>

  { /* STUDENT ONLY */ }
  <Route element={<ProtectedRoute allowedRoles={['student']} />}>
    <Route path="/student/dashboard" element={<StudentDashboardPage />} />
    ...
  </Route>

  { /* PROVIDER ONLY */ }
  <Route element={<ProtectedRoute allowedRoles={['Job_Provider']} />}>...</Route>

  { /* ADMIN ONLY */ }
  <Route element={<ProtectedRoute allowedRoles={['admin']} />}>...</Route>

  <Route path="*" element={<NotFoundPage />} />
</Routes>
```

Grouping matters: a `<Route element={<ProtectedRoute .../>}>` with children applies the guard to **every** child. Adding a student page inside that group protects it automatically.

`path="*" matches anything unmatched → NotFoundPage.`

:id route parameters

`/jobs/:id` matches `/jobs/abc123`, and the page reads it with `useParams()`:

```
JSX
const { id } = useParams();
```

ProtectedRoute

```
export default function ProtectedRoute({ allowedRoles }) {  
  const auth = useAuth();  
  const location = useLocation();  
  
  if (auth.isRestoring) return <Spinner label="Restoring your session..." />;  
  if (!auth.isAuthenticated) {  
    return <Navigate to={loginRoutes[allowedRoles[0]] || "/"} replace state={{ from: location.pathname }}  
    />;  
  }  
  if (!allowedRoles.includes(auth.user.role)) {  
    return <Navigate to={dashboardRoutes[auth.user.role] || "/"} replace />;  
  }  
  return <Outlet />;  
}
```

- **Not signed in** → redirect to the login page matching the first allowed role.
- **Signed in, wrong role** → redirect to *their own* dashboard (not the login page — they are logged in; they simply took a wrong turn).
- **Allowed** → `<Outlet />` renders the matched child route.

`replace` swaps the history entry instead of pushing one, so the browser Back button does not bounce the user into a redirect loop.

This is convenience, not security. A user who edits `sessionStorage` can make `ProtectedRoute` render the Admin dashboard. It will be an empty shell: every API call it makes returns 403, because the backend verifies the real token. The guard exists to give honest users sensible navigation, not to stop attackers.

Authentication on the frontend

Storage — `utils/authStorage.js`

The token and decoded claims are kept in `sessionStorage` under `ruwork.auth`.

```
export function readStoredAuth() {  
  const storage = getSessionStorage();  
  if (!storage) return null;  
  try {  
    const value = JSON.parse(storage.getItem(AUTH_STORAGE_KEY));  
    if (!value?.token || !value?.user?.role) return null;  
    return value;  
  } catch { storage.removeItem(AUTH_STORAGE_KEY); return null; }  
}
```

JS

`sessionStorage` vs `localStorage`: `sessionStorage` clears when the browser tab closes; `localStorage` persists indefinitely. RuWork chose the shorter-lived option deliberately.

Known limitation: both are readable by any JavaScript on the page, so a successful XSS attack could steal the token. A `HttpOnly` cookie would be invisible to JavaScript but requires CSRF protection and a different backend architecture. `tokenVersion` revocation limits the damage window. This trade-off is documented in `frontend/README.md` and `PROJECT_PLAN.md`.

Reading the token — `utils/token.js`

```
export function decodeAccessToken(token) {  
  try {  
    const [, payload] = token.split(".");  
    if (!payload) return null;  
    const decoded = JSON.parse(decodeBase64Url(payload));  
    if (!decoded?.sub || !decoded?.role) return null;  
    if (decoded.exp && decoded.exp * 1000 ≤ Date.now()) return null;  
    return decoded;  
  } catch { return null; }  
}
```

JS

This **decodes** the payload; it does **not verify** the signature (that needs the secret, which lives only on the server). The claims are used purely to decide which navigation to draw and which route group applies. `exp * 1000` converts JWT seconds to JavaScript milliseconds — a classic off-by-1000 bug source.

AuthContext

```
const [auth, setAuth] = useState(() => {  
  const stored = readStoredAuth();  
  const user = stored ? decodeAccessToken(stored.token) : null;  
  if (!stored || !user) { clearStoredAuth(); return null; }  
  return { token: stored.token, user };  
});
```

JSX

State is initialised **from storage**, so a page refresh in the same tab keeps you signed in. An expired or malformed token yields `null` from `decodeAccessToken`, and storage is cleared.

The context exposes `isAuthenticated`, `token`, `user`, `login`, `logout`, and `replaceToken`.

login calls the right service by role, decodes the returned token, verifies the role matches what was expected, stores, and sets state.

logout — note the order:

```
const logout = useCallback(async () => {  
  const role = auth?.user?.role;  
  if (role) await authService.logout(role).catch(() => {});  
  clearStoredAuth();  
  setAuth(null);  
}, [auth?.user?.role]);
```

JSX

Why call the server first, and why `.catch(() => {})`? *Server-side revocation is what actually kills the token (Backend §5.4). But if that request fails — offline, server down — we must **still** clear local state. Swallowing the error guarantees a failed network call can never leave someone apparently signed in.*

replaceToken adopts the fresh token returned by a successful password change, so the current device stays signed in while every other session is revoked.

Reacting to revocation elsewhere:

```
useEffect(() => {  
  function handleExpired() { setAuth(null); }  
  window.addEventListener(SESSION_EXPIRED_EVENT, handleExpired);  
  return () => window.removeEventListener(SESSION_EXPIRED_EVENT, handleExpired);  
}, []);
```

JSX

The Axios interceptor dispatches this event when the server rejects a token. React state updates, `ProtectedRoute` sees `isAuthenticated === false`, and the user is redirected to sign in — rather than sitting in a workspace where every request fails.

Axios and the shared API client

`services/api.js` creates **one** configured client used by every service.

```
const api = axios.create({
  baseURL: API_BASE_URL,           // "/api" in dev
  headers: { "Content-Type": "application/json" },
  timeout: 15000
});
```

JS

A timeout matters: without one, a hung request leaves a spinner forever.

Request interceptor — attaching the token

```
api.interceptors.request.use((config) => {
  const token = getStoredToken();
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

JS

An **interceptor** is a hook that runs on every request or response. Without this, every one of the ~60 service methods would have to attach the header itself — and one that forgot would produce a mysterious 401.

Security note: the token goes in a *header*, never in the URL. URLs end up in browser history, server access logs, and the `Referer` header sent to third parties.

Response interceptor — handling revoked sessions

```
const NON_SESSION_401_CODES = new Set(["CURRENT_PASSWORD_INVALID"]);

api.interceptors.response.use(
  (response) => response,
  (error) => {
    const status = error?.response?.status;
    const code = error?.response?.data?.code;
    if (status === 401 && !NON_SESSION_401_CODES.has(code) && getStoredToken()) {
      clearStoredAuth();
      window.dispatchEvent(new CustomEvent(SESSION_EXPIRED_EVENT, { detail: { code: code || "UNAUTHENTICATED" } }));
    }
    return Promise.reject(error);
  }
);
```

Why the exception list exists — a real bug caught in Phase 10 browser testing. The first version signed the user out on *any* 401. But PATCH /password returns 401 CURRENT_PASSWORD_INVALID when you mistype your current password — so a simple typo logged you out. The fix distinguishes “*your session is dead*” from “*that particular credential check failed inside a still-valid session*”. There is a regression test for it in `services/api.test.js`.

The general lesson: *a status code alone is not always enough context. The code field in RuWork’s error bodies exists precisely so the client can tell these cases apart.*

Safe error messages — `utils/apiError.js`

```
export function getApiError(error, fallback = "We could not complete that request. Please try again.") {
  if (!error?.response) { /* network error */ }
  const status = error.response.status;
  const data = error.response.data;
  return {
    message: typeof data?.error === "string" ? data.error : SAFE_FALLBACKS[status] || fallback,
    code: typeof data?.code === "string" ? data.code : "REQUEST_FAILED",
    status,
    retryAfterSeconds: Number(data?.retryAfterSeconds || error.response.headers?.["retry-after"]) || 0
  };
}
```

It distinguishes *no response at all* (network failure — “check your connection”) from *a response with an error status*, and falls back to a friendly message per status code. Every page uses it, so error handling is consistent.

The service layer

Every HTTP call lives in `services/`. Components never call Axios directly.

SERVICE	WRAPS
<code>authService</code>	Register, login (×3 roles), verify email, resend, forgot/reset/change password, logout
<code>jobService</code>	Browse, details, provider CRUD
<code>applicationService</code>	Apply, list, withdraw, cancel, accept, decline, complete
<code>reviewService</code>	Create/delete own review, job reviews, provider reviews, admin reviews
<code>messageService</code>	Conversations, thread, send, unread count
<code>notificationService</code>	List, mark read, mark all read, unread count
<code>profileService</code>	Student and provider profile read/update
<code>dashboardService</code>	Student, provider, and admin dashboards; job history
<code>adminService</code>	Registrations, accounts, jobs, review moderation, settings, audits

A typical method:

```

async moderateJob(id, status, reason = "") {
  const { data } = await api.patch(`/admin/jobs/${encodeURIComponent(id)}/moderation`,
    { status, ...(reason ? { reason } : {}) });
  return data;
}

```

JS

Two details: `encodeURIComponent` prevents an id containing `/` or `?` from corrupting the URL, and the conditional spread `...(reason ? { reason } : {})` omits `reason` entirely when empty — important because the backend's `assertOnlyFields` rejects unexpected keys, and an empty-string `reason` would fail the 5-character minimum.

Why a service layer?

1. **One place per endpoint.** If a URL changes you edit one line, not fifteen pages.
2. **Trivially mockable.** Tests do `vi.mock("../services/adminService")` and control every response without a network.
3. **Consistent shape.** Services unwrap `data` so pages work with plain objects.
4. **Pages stay about UI.** A page manages loading/error/empty state; it does not know about HTTP verbs.

Pages vs components

	PAGE	COMPONENT
Location	src/pages/**	src/components/**
Mapped to a URL?	Yes	No
Fetches data?	Usually	Rarely
Reused?	No	Yes
Example	AdminJobsPage.jsx	Button.jsx

A **page** owns a screen: it fetches, holds loading/error state, and composes components. A **component** is a reusable building block that receives props and renders.

The components/ subfolders group by domain: admin/, applications/, auth/, common/, jobs/, layout/, reviews/, workspace/.

Shared components

components/common/ — the design-system primitives.

COMPONENT	PURPOSE	NOTES
Button.jsx	All buttons and button-styled links	variant (primary/secondary/danger), isLoading, and as={Link} to render as a router link
FormField.jsx	Labelled text input	Wires htmlFor/id, aria-invalid, and aria-describedby automatically
PasswordField.jsx	Password input	Adds a show/hide toggle with a correct aria-label
SelectField.jsx	Labelled <select>	Takes an options array
TextareaField.jsx	Labelled textarea	Used for notes and moderation reasons
Alert.jsx	Inline message	tone="error" → role="alert"; otherwise role="status"
Modal.jsx	Focus-trapped dialog	Escape to close, focus contained, restored on close
Spinner.jsx	Loading indicator	role="status" with a text label
Logo.jsx	RuWork wordmark	
ScrollToTop.jsx	Resets scroll on navigation	Renders nothing
PaymentInformationCard.jsx	“RuWork does not process payments” notice	Shown on Job and Application details

Why FormField instead of a raw <input>?

```
const messageId = error ? `${id}-error` : helper ? `${id}-helper` : undefined;
<input id={id} aria-invalid={Boolean(error)} aria-describedby={messageId} ... />
{(error || helper) && <p id={messageId} ...>{error || helper}</p>}
```

JSX

Every field automatically gets a correctly associated label and error message. If each page hand-rolled its inputs, some would forget htmlFor, and screen-reader users would hit unlabelled fields. Centralising makes accessibility the default rather than something to remember.

Student flows

PAGE	ROUTE	WHAT IT DOES
LoginPage (role="student")	/login/student	Validates the @ruh.ac.lk address client-side, calls login, routes EMAIL_NOT_VERIFIED / ACCOUNT_PENDING / ACCOUNT_REJECTED to dedicated state pages
StudentRegistrationPage	/register/student	Full registration form with the fixed university value shown read-only
StudentDashboardPage	/student/dashboard	Summary counts, recent applications, suggested jobs
FindJobsPage	/jobs	Search, filter, sort, paginate — state synchronised to the URL
JobDetailsPage	/jobs/:id	Full details, ratings, paginated reviews, the Apply flow
MyApplicationsPage	/student/applications	Status-filtered list
ApplicationDetailsPage	/student/applications/:id	Prices, withdraw/cancel confirmations, review form when completed
JobHistoryPage	/student/job-history	Terminal applications, paginated
MessagesPage	/student/messages, /student/messages/:applicationId	Shared inbox
NotificationsPage	/student/notifications	Shared notifications
StudentProfilePage	/student/profile	Editable fields only; email/university read-only
ChangePasswordPage	/account/password	Shared by all three roles

URL synchronisation on Find Jobs means filters live in the query string (/jobs?category=Tutoring&page=2). The result is shareable and bookmarkable, Back/Forward work naturally, and a refresh preserves the search.

The Apply flow (components/applications/ApplyToJob.jsx) is availability- and role-aware: signed-out visitors get a prompt to log in, providers see it disabled, closed or expired jobs block it, and an existing application links to it instead. All of this is *usability* — the backend re-checks every condition.

Provider flows

PAGE	ROUTE	WHAT IT DOES
ProviderRegistrationPage	/register/provider	Company + contact fields, 300-char description limit
ProviderDashboardPage	/provider/dashboard	Owned job/application aggregates, recent activity
MyJobsPage	/provider/jobs	Status filters; View / Edit / Publish / Reopen / Close / Archive (modal-confirmed)
JobFormPage	/provider/jobs/new, /provider/jobs/:id/edit	Multi-step editor with preview
ApplicantsPage	/provider/jobs/:jobId/applications	Applicants for one job
ProviderApplicationDetailsPage	/provider/applications/:id	Accept (with agreed price) / decline / complete
CompanyProfilePage	/provider/profile	Editable company data; email read-only
ProviderReviewsPage	/provider/reviews	Overall rating, view-only list

The **multi-step job form** keeps all values in local state across Basics → Skills & Scope → Work Details → Pricing → Description → Preview, then submits **once**. Preview creates no database record — it renders the same `JobPreview` component the public page uses, so what you see is what will be published.

Pricing is conditional: choosing `hourly` shows only the rate input, `fixed` only the budget input — mirroring the backend's `pre("validate")` rule.

Admin workspace

Seven navigation entries, plus detail views reached from them.

PAGE	ROUTE	PURPOSE
AdminDashboardPage	/admin/dashboard	Account/job/application/review counts, message counts only , six recent audits
RegistrationReviewsPage	/admin/registrations	Paginated queue with type/status filters
RegistrationDetailsPage	/admin/registrations/:type/:id	Full record; modal-confirmed approve/reject
AdminAccountsPage	/admin/students, /admin/providers	One component, <code>type</code> prop; search/filter/paginate; suspend/restore
AdminAccountDetailsPage	/admin/students/:id, /admin/providers/:id	Sanitised detail
AdminJobsPage	/admin/jobs	Lifecycle/moderation/archive filters; hide/restore
AdminJobDetailsPage	/admin/jobs/:id	Inspection with the owning provider
AdminReviewsPage	/admin/reviews	Filter, hide/restore, delete
AdminSettingsPage	/admin/settings	Three toggles + settings-scoped audit history
AdminAuditTrailPage	/admin/audits	Full read-only trail, action + record-type filters

Reused admin components (components/admin/):

- `ModerationBadge.jsx` — consistent status pill.
- `ModerationDialog.jsx` — the confirmation dialog. It requires a reason (≥ 5 characters) for destructive-direction actions and disables the confirm button until one is entered, mirroring the backend's `moderationReason(value, { required: true })`.
- `AdminPagination.jsx` — renders nothing when `pages ≤ 1`.

The Audit Trail is deliberately read-only — no edit or delete controls exist, matching the immutable model. A test asserts this.

The Settings page states in the UI that JWT secrets, MongoDB credentials, SMTP credentials, and Admin passwords are environment-managed and intentionally unavailable there — so nobody goes looking for a place to type them.

Messaging UI

One `MessagesPage.jsx` serves both roles, at `/student/messages` and `/provider/messages`.

- **Desktop:** two columns — conversation list on the left, thread on the right.
- **Mobile:** one column at a time. `/.../messages` shows the list; tapping a conversation navigates to `/.../messages/:applicationId`, which shows the thread with a Back control.

The `:applicationId` parameter drives everything: conversations are grouped by application, exactly as the backend models them (*Backend §15.1*).

Unread behaviour: opening a thread marks received messages read, then dispatches a `COMMUNICATION_UNREAD_EVENT` (`utils/communication.js`) that tells `AppHeader` to refresh its badges immediately, instead of waiting for a navigation.

Contact sharing: the Student composer has an explicit checkbox. It sends only `includeContactNumber: true` — never a number. The backend reads the number from the authenticated profile. The control is not rendered for Providers.

Notifications UI

One `NotificationsPage.jsx` for both roles.

- Unread items are visually distinct **and** carry a semantic label, so the difference is not conveyed by colour alone.
- Individual “mark read” and a “mark all read” action.
- Contextual navigation: each notification links to the right destination, built **only** from validated type→route mappings plus stored ids — never from a server-supplied URL.

Why not accept a URL from the server? *Rendering a link target from data is how open-redirect and injection bugs happen. Mapping a known type to a known route keeps the set of possible destinations finite and auditable.*

Header badges (`components/layout/AppHeader.jsx` → `UnreadBadge`):

```
return count > 0 ? (  
  <span ... aria-label={` ${count} unread ${label.toLowerCase()} `}>{count > 99 ? "99+" : count}</span>  
) : null;
```

JSX

Rendered only when positive, capped at 99+, and given an `aria-label` so a screen reader announces “3 unread messages” rather than a bare “3”.

Styling with Tailwind

Tailwind is **utility-first**: instead of writing CSS, you compose small classes in the markup.

```
<div className="mt-4 flex flex-col gap-4 rounded-2xl bg-white p-6 sm:flex-row sm:items-center">
```

JSX

mt-4 = margin-top; flex flex-col = flexbox column; gap-4 = spacing between children; rounded-2xl, bg-white, p-6 = radius, background, padding.

The design tokens

src/index.css defines RuWork's palette with Tailwind v4's @theme:

```
@import "tailwindcss";
@theme {
  --color-brand-600: #5b36d6;
  --color-ink-950: #131625;
  --color-surface: #f8f8fc;
  --font-sans: "Inter", "Segoe UI", system-ui, sans-serif;
  --shadow-soft: 0 18px 60px -30px rgba(55, 37, 128, 0.3);
}
```

CSS

These generate bg-brand-600, text-ink-950, bg-surface, shadow-soft, and so on. Defining tokens once means the indigo-purple identity is consistent everywhere and changeable in one place.

Responsive prefixes — mobile-first

PREFIX	APPLIES FROM	TYPICAL DEVICE
(none)	0px	Mobile
sm:	640px	Large phone / small tablet
md:	768px	Tablet
lg:	1024px	Laptop
xl:	1280px	Desktop

Mobile-first means unprefixed classes are the *mobile* style, and prefixes add larger-screen overrides.


```
<div className="grid gap-4 sm:grid-cols-2 xl:grid-cols-4">
```

J S X

One column on a phone, two from 640px, four from 1280px.

```
<nav className="hidden items-center md:flex">      {/* desktop nav */}
<button className="... md:hidden">                {/* hamburger, mobile only */}
```

J S X

Why mobile-first? *Most students browse on a phone. Starting from the constrained layout and adding space for larger screens produces designs that work on small screens by default — rather than desktop designs awkwardly squeezed down.*

Verified: every RuWork page was checked at 1440×900 and 390×844 with **no horizontal overflow** — nothing forces the user to scroll sideways.

Accessibility

Labels

Every input has a real `<label htmlFor={id}>` paired with a matching `id`. Wired centrally in `FormField`, `PasswordField`, `SelectField`, and `TextAreaField`, so it cannot be forgotten.

Why not placeholder as a label? Placeholders vanish once typing starts, are often too low-contrast, and are inconsistently announced by screen readers. RuWork uses placeholders only as examples (`name@ruh.ac.lk`) alongside a real label.

aria-invalid and aria-describedby

```
<input id={id} aria-invalid={Boolean(error)} aria-describedby={messageId} />
{(error || helper) && <p id={messageId}>{error || helper}</p>}
```

JSX

Red borders alone are invisible to a screen-reader user (and to many colour-blind users). `aria-invalid` announces the field as invalid; `aria-describedby` links it to the message so the *reason* is read out too. Verified in the browser during Phase 10.

aria-live

```
<div className="fixed top-4 right-4 ..." aria-live="polite">
```

JSX

Toasts appear in a live region, so screen readers announce them without the user hunting for the change. `polite` waits for a pause rather than interrupting.

Spinner uses `role="status"`; Alert uses `role="alert"` for errors (assertive) and `role="status"` otherwise.

Focus management

`Modal.jsx` traps focus inside the dialog, closes on Escape, and restores focus to the trigger on close — so keyboard users are never stranded behind an invisible dialog.

Skip link

```
<a href="#main-content" className="sr-only focus:not-sr-only focus:absolute focus:top-3 ..." >
  Skip to main content
</a>
<div id="main-content" tabIndex={-1}>
```

JSX

`sr-only` hides it visually but keeps it available to screen readers; `focus:not-sr-only` reveals it when tabbed to. `tabIndex={-1}` on the target is essential — a plain `<div>` cannot receive focus, so without it the skip link would move the scroll position but not the keyboard focus, and the next Tab would return to the top of the navigation.

Keyboard and semantics

Real `<button>` and `<a>` elements throughout (never a clickable `<div>`), so Enter/Space and focus rings work for free. Forms are real `<form>` elements with a submit button, so Enter submits.

Honest scope: accessibility was verified by structural and keyboard checks in a real browser, **not** by a full screen-reader session or an automated WCAG audit. No formal conformance level is claimed.

Lazy loading and performance

The problem

Everything imported at the top of `App.jsx` lands in one bundle, downloaded before the first paint. That included the entire Admin workspace — which a student will never open.

The fix

```
import { Suspense, lazy } from "react";

// eager - the public entry a first-time visitor needs
import LandingPage from "../pages/public/LandingPage";
import LoginPage from "../pages/auth/LoginPage";
import FindJobsPage from "../pages/jobs/FindJobsPage";
import JobDetailsPage from "../pages/jobs/JobDetailsPage";

// lazy - fetched only when the route is visited
const AdminDashboardPage = lazy(() => import("../pages/admin/AdminDashboardPage"));
const StudentDashboardPage = lazy(() => import("../pages/student/StudentDashboardPage"));
...
```

`lazy(() => import("../..."))` tells Vite to emit a separate file and fetch it on demand. Because the file is not needed until that route renders, `Suspense` supplies a fallback meanwhile:

```
<Suspense fallback={<Spinner label="Loading RuWork..." />}>
  <Routes>...</Routes>
</Suspense>
```

Without `Suspense`, React throws — a lazy component *must* have a boundary above it.

What stays eager, and why

The landing page, login, job browse, and job details load immediately. They are what a first-time visitor sees; lazy-loading them would add a spinner to the very first impression. Everything behind authentication is split out.

The measured result

	BEFORE	AFTER
Main chunk	517.68 kB	376.92 kB
Gzipped	143.73 kB	119.11 kB
Vite 500 kB advisory	Shown	Gone

Roughly 27% smaller, and the remaining pages arrive as small chunks only when needed. No behaviour changed — this is purely a delivery improvement.

Frontend testing

The tools

- **Vitest** — the test runner, sharing Vite’s config so imports and JSX work identically.
- **React Testing Library (RTL)** — renders components and queries them **the way a user would**.
- **jsdom** — a browser-like DOM in Node, so there is no real browser to drive.

The guiding principle

RTL encourages querying by what a user perceives:

```
screen.getByLabelText("University email")
screen.getByRole("button", { name: "Change password" })
screen.getByText("Password changed successfully.")
```

JSX

...rather than by CSS class or internal state.

Why? *A test that asserts on a class name breaks when you restyle, even though nothing user-visible changed. A test that asserts a button labelled “Change password” exists keeps passing through restyles and fails if the button disappears — which is exactly when you want to know.*

Shared setup

`src/test/setup.js` clears `sessionStorage` before each test and cleans up the DOM after, so tests cannot leak state into each other.

`src/test/renderWithProviders.jsx` wraps a component in `MemoryRouter`, `ToastProvider`, and `AuthProvider`, and can pre-authenticate a role:

```
renderWithProviders(<AdminAccountsPage type="students" />, { role: "admin" });
```

JSX

`MemoryRouter` keeps routing in memory (no real URL bar), and `{ route:`

`"/admin/students/student-1" }` sets the starting location for pages using `useParams`.

Mocking services

```
vi.mock("../services/adminService", () => ({
  adminService: { getAccounts: vi.fn(), moderateAccount: vi.fn(), ... }
}));

adminService.getAccounts.mockResolvedValue({ accounts: [student], pagination });
```

JSX

Replacing the service means tests are fast, deterministic, need no backend, and can force error paths on demand:

```
adminService.getAudits.mockRejectedValue({ response: { data: { error: "Audit trail unavailable" } } });
```

JSX

A representative test

```
it("requires a reason, suspends a Student, and shows success feedback", async () => {
  adminService.getAccounts.mockResolvedValue({ accounts: [student], pagination: { ..., pages: 1 } });
  adminService.moderateAccount.mockResolvedValue({ message: "Student suspended successfully", account: { ...
} });
  renderWithProviders(<AdminAccountsPage type="students" />, { role: "admin" });

  fireEvent.click(await screen.findByRole("button", { name: "Suspend" }));
  const confirm = screen.getAllByRole("button", { name: "Suspend" }).at(-1);
  expect(confirm).toBeDisabled(); // reason required
  fireEvent.change(screen.getByLabelText("Reason"), { target: { value: "Policy breach" } });
  fireEvent.click(confirm);
  await waitFor(() => expect(adminService.moderateAccount)
    .toHaveBeenCalled("students", "student-1", "suspended", "Policy breach"));
  expect(await screen.findByText("Student suspended successfully")).toBeInTheDocument();
});
```

JSX

Note `findBy*` (async, waits) versus `getBy*` (sync, throws immediately) — and `waitFor` for assertions that need a re-render.

The suites

FILE	COVERS
App.test.jsx	Route map and protection
pages/Phase6Pages.test.js	Dashboards, profiles, registration reviews
pages/Phase7Reviews.test.jsx	Review creation, listings, moderation
pages/Phase8Communication.test.jsx	Messages and notifications
pages/Phase9Admin.test.js	Full admin workspace
pages/Phase10Hardening.test.js	Password lifecycle, sign-out, revocation
services/api.test.js	Base URL and the 401 interceptor rules
Component/page tests	JobCard, ApplyToJob, ProtectedRoute, MyJobsPage, JobFormPage, FindJobsPage, JobDetailsPage, VerifyEmailPage, AccountStatePage, LandingPage, validation

Current totals

METRIC	VALUE
Frontend tests	94 / 94 passing
Test files	21
ESLint	Passes, no errors or warnings
Production build	Succeeds; main chunk $\approx$ 377 kB
npm audit	0 vulnerabilities

File-by-file reference

Entry, config, routing

FILE	MAIN RESPONSIBILITY	IMPORTANT EXPORTS	USED BY
src/main.jsx	Mounts React; nests Router/Toast/Auth providers	—	index.html
src/App.jsx	Route map, role groups, skip link, Suspense	App	main.jsx
vite.config.js	Plugins, dev /api proxy, Vitest config	—	Vite
src/index.css	Tailwind import + design tokens	—	main.jsx
eslint.config.js	Lint rules	—	npm run lint

Context, hooks, utils

FILE	RESPONSIBILITY	IMPORTANT EXPORTS	USED BY
context/AuthContext.jsx	Session state; login/logout/replaceToken; expiry listener	AuthProvider	main.jsx
context/authContextValue.js	The context object	default	AuthContext, useAuth
context/ToastContext.jsx	Toast queue in an aria-live region	ToastProvider	main.jsx
context/toastContextValue.js	The context object	default	ToastContext, useToast
hooks/useAuth.js	Guarded auth consumer	default	Most pages
hooks/useToasts.js	Guarded toast consumer	default	Pages with feedback
utils/authStorage.js	sessionStorage read/write/clear	readStoredAuth, storeAuth, clearStoredAuth, getStoredToken, AUTH_STORAGE_KEY	api.js, AuthContext
utils/token.js	Decode JWT payload (no verification)	decodeAccessToken	AuthContext
utils/apiErrors.js	Safe user-facing error messages	getApiError	Every page
utils/validation.js	Client-side form rules	isRuhunaEmail, isBasicEmail, getPasswordError, isValidOptionalUrl	Auth and profile pages
utils/jobOptions.js	Job dropdown options, date formatting	formatJobDate, option lists	Job and admin pages
utils/applicationOptions.js	Application status labels/options	—	Application pages
utils/communication.js	Unread-refresh event name	COMMUNICATION_UNREAD_EVENT	AppHeader, messaging

Services

FILE	RESPONSIBILITY	USED BY
services/api.js	Shared Axios client; token + 401 interceptors	Every service
services/authService.js	Registration, login, verification, password, logout	Auth pages, AuthContext
services/jobService.js	Job browse and provider CRUD	Job pages
services/applicationService.js	Application lifecycle	Application pages
services/reviewService.js	Reviews (student/public/provider/admin)	Review pages
services/messageService.js	Conversations and sending	MessagesPage, AppHeader
services/notificationService.js	Notifications and counts	NotificationsPage, AppHeader
services/profileService.js	Profile read/update	Profile pages
services/dashboardService.js	Dashboards and job history	Dashboard pages
services/adminService.js	All admin operations	Admin pages

Layout and shared components

FILE	RESPONSIBILITY
components/layout/AppHeader.jsx	Role-aware nav, unread badges, Password link, sign-out; mobile toggle
components/layout/PublicHeader.jsx / PublicFooter.jsx	Public chrome
components/layout/AuthShell.jsx	Split-screen auth layout
components/auth/ProtectedRoute.jsx	Route guard
components/auth/RoleSelectionModal.jsx	Student / Provider / Admin picker
components/auth/RegistrationSection.jsx	Grouped form sections
components/auth/ResendVerificationForm.jsx	Resend with cooldown
components/common/*	Design-system primitives — see §8
components/jobs/*	JobCard, JobCardSkeleton, JobPreview, JobStatusBadge, SkillInput
components/applications/*	ApplyToJob, StudentApplicationActions, ProviderApplicationActions, ApplicationStatusBadge
components/reviews/*	StarRatingInput, ReviewCard, RatingSummary, JobReviewsSection, StudentReviewActions
components/admin/*	ModerationBadge, ModerationDialog, AdminPagination
components/workspace/*	WorkspaceStatCard, AccountStatusBadge

Pages

FOLDER	PAGES
pages/public/ 	LandingPage
pages/auth/ 	LoginPage, StudentRegistrationPage, ProviderRegistrationPage, VerifyEmailPage, AccountStatePage, ForgotPasswordPage, ResetPasswordPage, ChangePasswordPage
pages/jobs/ 	FindJobsPage, JobDetailsPage
pages/student/ 	StudentDashboardPage, MyApplicationsPage, ApplicationDetailsPage, JobHistoryPage, StudentProfilePage
pages/provider/ 	ProviderDashboardPage, MyJobsPage, JobFormPage, ApplicantsPage, ProviderApplicationDetailsPage, CompanyProfilePage, ProviderReviewsPage
pages/admin/ 	AdminDashboardPage, RegistrationReviewsPage, RegistrationDetailsPage, AdminAccountsPage, AdminAccountDetailsPage, AdminJobsPage, AdminJobDetailsPage, AdminReviewsPage, AdminSettingsPage, AdminAuditTrailPage
pages/messages/ 	MessagesPage (both roles)
pages/notifications/ 	NotificationsPage (both roles)
pages/ 	NotFoundPage

Frontend common questions

Why do components not call Axios directly? See §6: one place per endpoint, easy mocking in tests, and pages that stay focused on UI.

Why use Context instead of passing props? `AppHeader` sits several layers below `App` and needs `auth`. Without context every component in between would have to accept and forward an `auth` prop it does not use (“prop drilling”). Context lets any descendant read it directly.

Why is `ProtectedRoute` not security? It runs in the user’s browser, which the user controls. It produces correct navigation for honest users; the backend produces correct *authorization* for everyone.

Why store the token in `sessionStorage` at all? Something must persist it across a page refresh. `sessionStorage` is narrower than `localStorage` (cleared when the tab closes). The XSS exposure is real and documented; server-side revocation limits the damage window.

Why does `logout` call the server before clearing local state? Clearing local state only removes this browser’s copy. The token stays valid until it expires unless the server increments `tokenVersion`. The `.catch(() => {})` guarantees local state is cleared even if that call fails.

Why did the 401 interceptor need an exception list? Because `PATCH /password` returns 401 `CURRENT_PASSWORD_INVALID` for a mistyped current password — not an expired session. Without the exception, a typo signed you out. See §5.

Why do effects use an active flag? To ignore a response that arrives after the component has unmounted, avoiding a state update on a dead component.

Why lazy-load routes? So a student never downloads the Admin workspace. It cut the main bundle by ~27%.

NEXT: CODE GLOSSARY · REQUEST & DATA FLOWS

